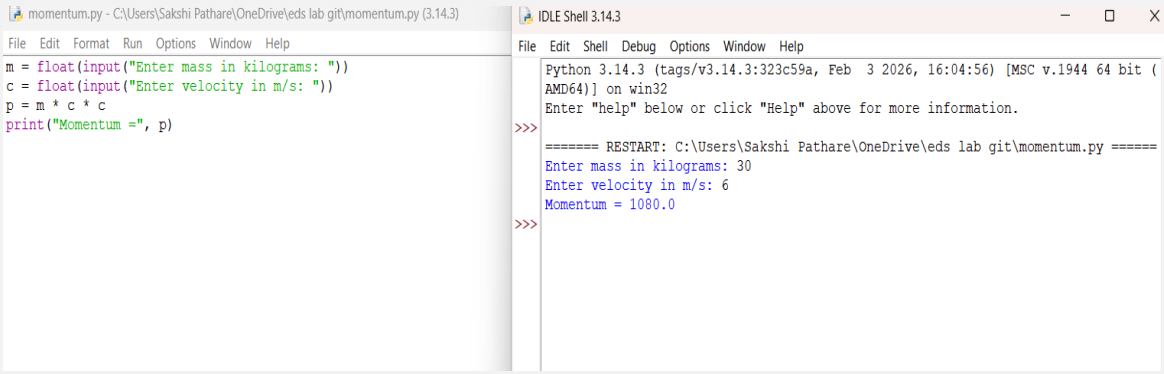
1.To accept an object mass in kilograms and velocity in meters per second and display its momentum.

Momentum is calculated as e=mc2 where m is the mass of the object and c is its velocity.

```
m = float(input("Enter mass in kilograms: "))

c = float(input("Enter velocity in m/s: "))

p = m * c * c

print("Momentum =", p)
```

OUTPUT:



2. Write a Python program for following conditions.

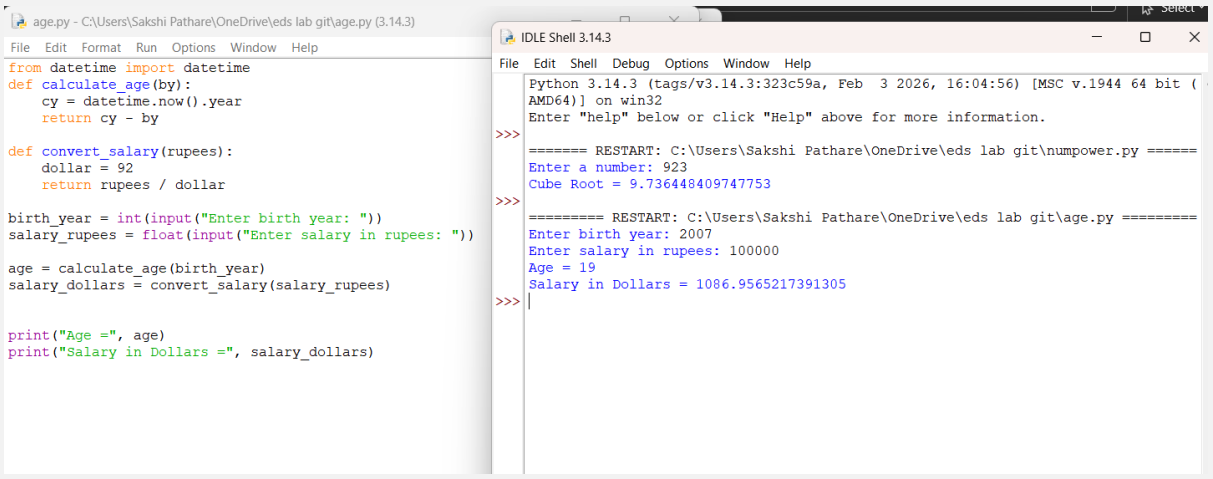
- If n is single digit print square of it.
- If n is two digit print square root of it.
- If n is three digit print cube root of it.

```
import math
n = int(input("Enter a number: "))
digits = len(str(n))
if digits == 1:
   print("Square =", n ** 2)
elif digits == 2:
   print("Square Root =", math.sqrt(n))
elif digits == 3:
   print("Cube Root =", n ** (1/3))
else:
```

```
print("Number is not single, two, or three digit")
```

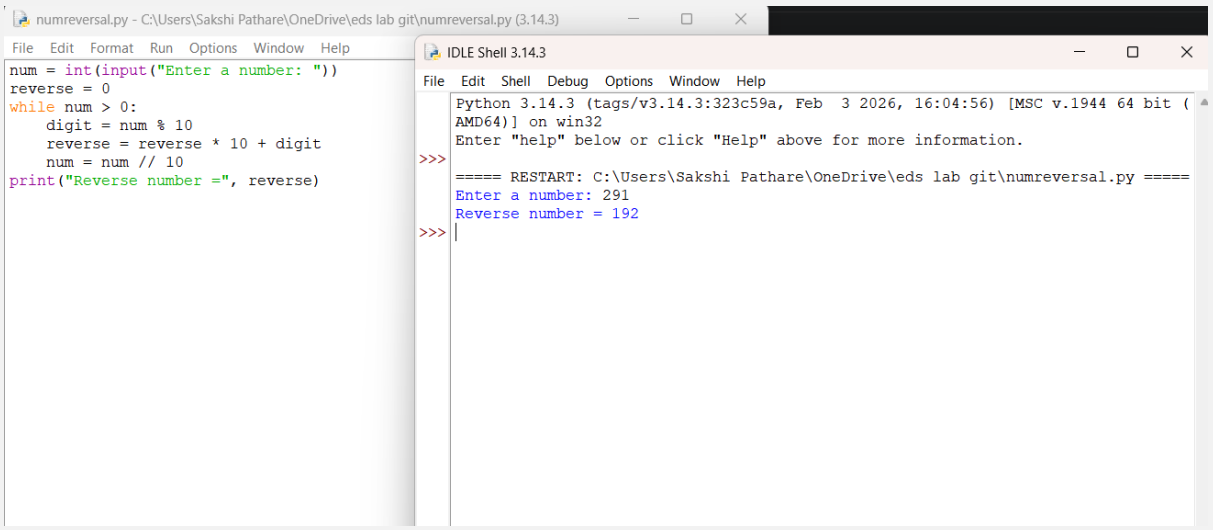
3. Read the birth date and salary in rupees of employees. Perform data transformation for birthdate to age and also salary which is in rupees to salary in dollars using functions.

```
from datetime import datetime

def calculate_age(by):
    cy = datetime.now().year
    return cy - by

def convert_salary(rupees):
    dollar = 92
    return rupees / dollar

birth_year = int(input("Enter birth year: "))

salary_rupees = float(input("Enter salary in rupees: "))

age = calculate_age(birth_year)

salary_dollars = convert_salary(salary_rupees)

print("Age =", age)

print("Salary in Dollars =", salary_dollars)
```



4. Print the reverse number of a given number.

```
num = int(input("Enter a number: "))
reverse = 0
while num > 0:
    digit = num % 10
    reverse = reverse * 10 + digit
    num = num // 10
print("Reverse number =", reverse)
```



Lab Assignment:

1. To accept students five courses marks and compute his/her result. Student is passing if he/she scores marks equal to and above 40 in each course.

If student scores aggregate greater than 75 percentage, then the grade is distinction.

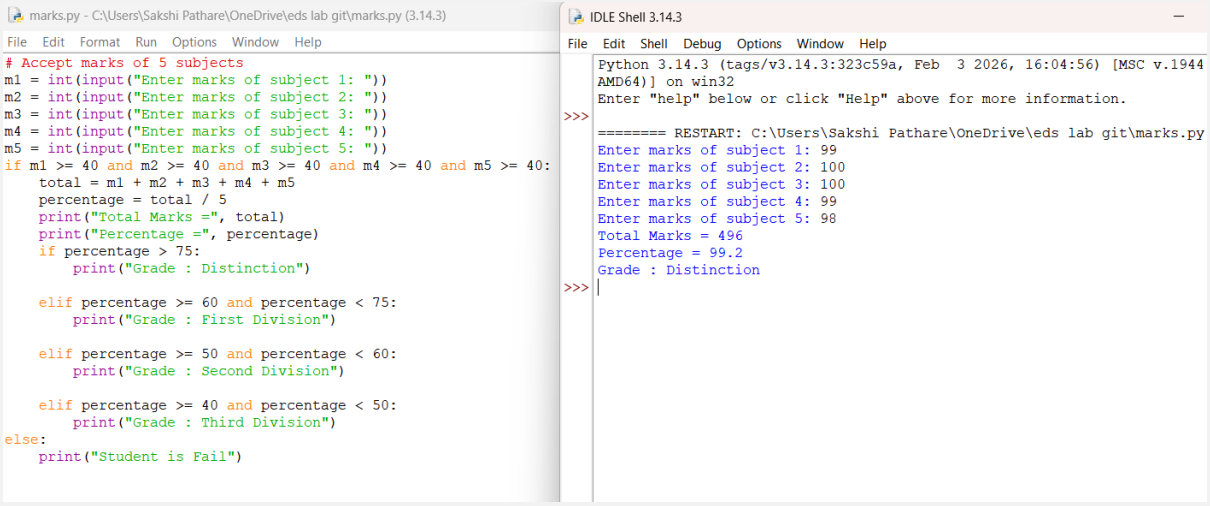
If aggregate is greater than or equal to 60 and less than 75 then the grade if first division.

If aggregate is greater than or equal 50 and less than 60, then the grade is second division.

If aggregate is greater than or equal 40 and less than 50, then the grade is third division.

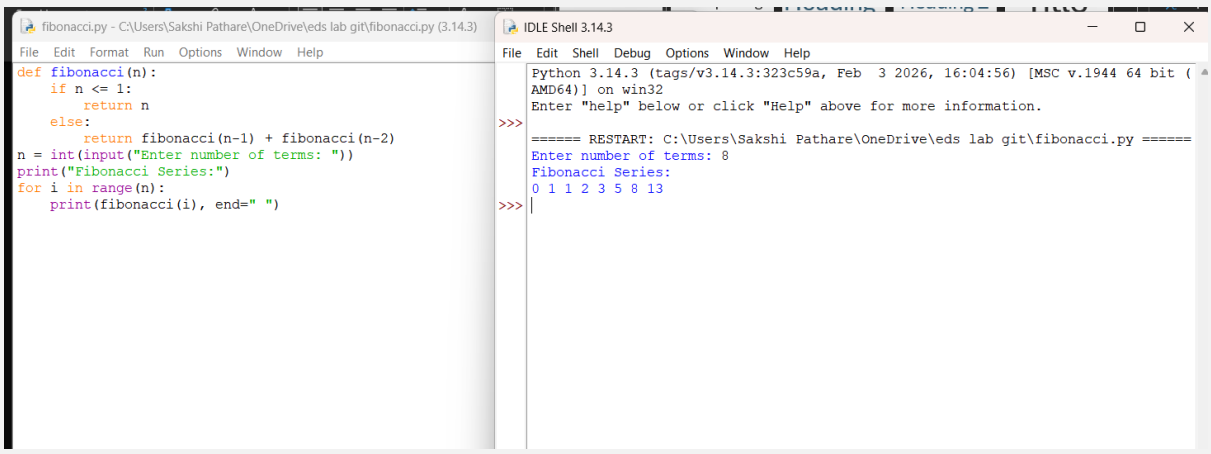
```
# Accept marks of 5 subjects
m1 = int(input("Enter marks of subject 1: "))
m2 = int(input("Enter marks of subject 2: "))
m3 = int(input("Enter marks of subject 3: "))
m4 = int(input("Enter marks of subject 4: "))
m5 = int(input("Enter marks of subject 5: "))
if m1 >= 40 and m2 >= 40 and m3 >= 40 and m4 >= 40 and m5 >= 40:
   total = m1 + m2 + m3 + m4 + m5
   percentage = total / 5
   print("Total Marks =", total)
   print("Percentage =", percentage)
   if percentage > 75:
      print("Grade : Distinction")

   elif percentage >= 60 and percentage < 75:
      print("Grade : First Division")

   elif percentage >= 50 and percentage < 60:
      print("Grade : Second Division")

   elif percentage >= 40 and percentage < 50:
      print("Grade : Third Division")
else:
```

print("Student is Fail")



2. Solve the Fibonacci sequence using recursive function in Python.

```
def fibonacci(n):
    if n <= 1:
        return n
    else:
        return fibonacci(n-1) + fibonacci(n-2)
n = int(input("Enter number of terms: "))
print("Fibonacci Series:")
for i in range(n):
    print(fibonacci(i), end=" ")
```



3. Write a Python program to print different patterns.

```
#increaasing star
n=int(input("enter number of rows:"))
for i in range(n):
   for j in range(i+1):
      print("*",end="")
   print()
#decreasing star
n1=int(input("enter number of rows:"))
for i in range(n1):
   for j in range(i,n1):
      print("*",end="")
   print()

#right sided triangle
n2=int(input("enter number of rows:"))
for i in range(n2):
   for j in range(i,n2):
      print(" ",end="")
   for j in range(i+1):
      print("*",end="")
   print()

#right handed decreasing
n3=int(input("enter number of rows:"))
for i in range(n3):
   for j in range(i+1):
      print(" ",end="")
```

```
for j in range(i,n3):
   print("*",end="")
print()
```

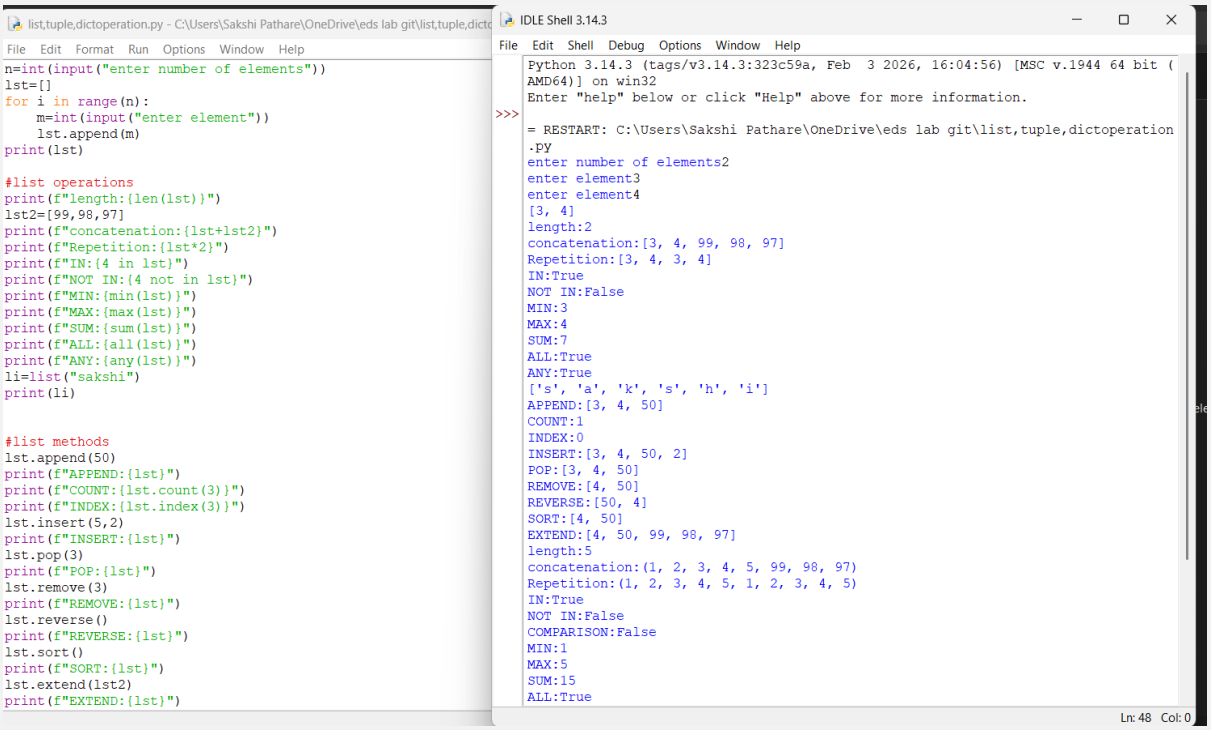
Practice Lab Assignment:

1. **Perform all List Operations, Dictionary Operations.**

```
n=int(input("enter number of elements"))
lst=[]
for i in range(n):
   m=int(input("enter element"))
   lst.append(m)
print(lst)

#list operations
print(f"length:{len(lst)}")
lst2=[99,98,97]
print(f"concatenation:{lst+lst2}")
print(f"Repetition:{lst*2}")
print(f"IN:{4 in lst}")
print(f"NOT IN:{4 not in lst}")
print(f"MIN:{min(lst)}")
print(f"MAX:{max(lst)}")
print(f"SUM:{sum(lst)}")
print(f"ALL:{all(lst)}")
print(f"ANY:{any(lst)}")
li=list("sakshi")
print(li)


#list methods
lst.append(50)
print(f"APPEND:{lst}")
print(f"COUNT:{lst.count(3)}")
print(f"INDEX:{lst.index(3)}")
lst.insert(5,2)
print(f"INSERT:{lst}")
lst.pop(3)
print(f"POP:{lst}")
lst.remove(3)
```

```
print(f"REMOVE:{lst}")
lst.reverse()
print(f"REVERSE:{lst}")
lst.sort()
print(f"SORT:{lst}")
lst.extend(lst2)
print(f"EXTEND:{lst}")

#TUPLE OPERATIONS
tuple=(1,2,3,4,5)
print(f"length:{len(tuple)}")
tup2=(99,98,97)
print(f"concatenation:{tuple+tup2}")
print(f"Repetition:{tuple*2}")
print(f"IN:{4 in tuple}")
print(f"NOT IN:{4 not in tuple}")
print(f"COMPARISON:{tuple==tup2}")
print(f"MIN:{min(tuple)}")
print(f"MAX:{max(tuple)}")
print(f"SUM:{sum(tuple)}")
print(f"ALL:{all(tuple)}")
print(f"ANY:{any(tuple)}")


#TUPLE methods

print(f"COUNT:{tuple.count(3)}")
print(f"INDEX:{tuple.index(3)}")

#dictionary operations

dict={1,2,3,4,5}
print(f"length:{len(dict)}")
print(f"COPY:{dict.copy()}")
print(f"GET:{dict.get(4)}")
print(f"IN:{4 in lst}")
print(f"NOT IN:{4 not in lst}")
print(f"MIN:{min(lst)}")
print(f"MAX:{max(lst)}")
print(f"SUM:{sum(lst)}")
print(f"ALL:{all(lst)}")
```

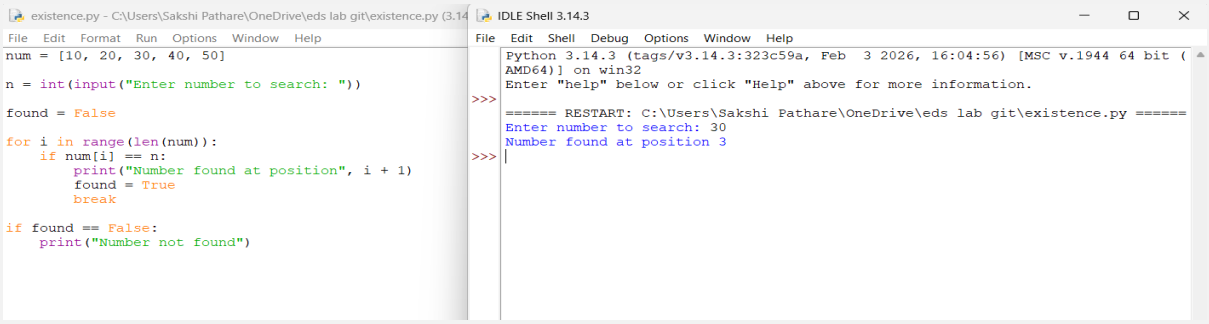
```
print(f"ANY:{any(lst)}")
print(f"second:{str(dict)}")
print(f"CLEAR:{dict.clear()}")
li=list("sakshi")
print(li)
```



Lab Assignment:

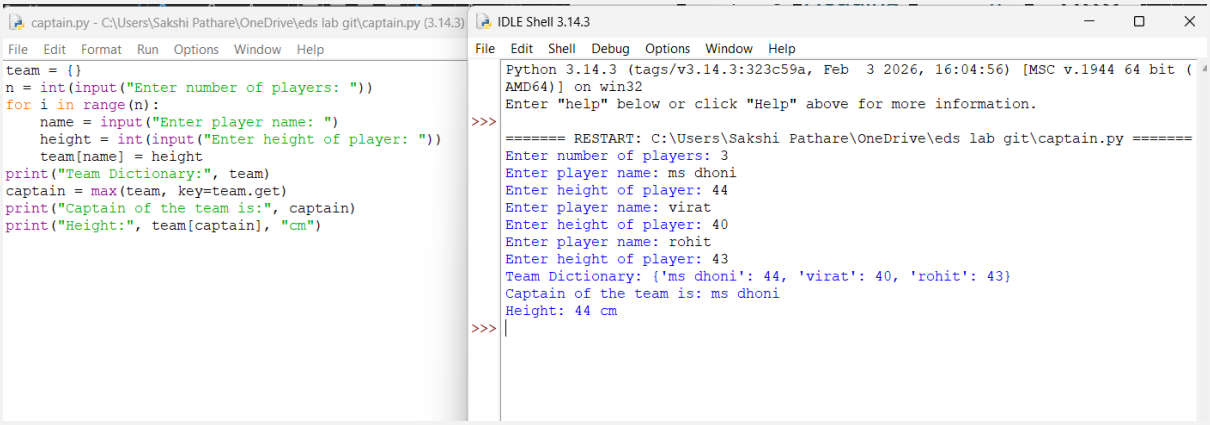
1. **Select the number from the entered list and find its position in Python (use Linear Search).**

```
num = [10, 20, 30, 40, 50]
n = int(input("Enter number to search: "))
found = False
for i in range(len(num)):
  if num[i] == n:
    print("Number found at position", i + 1)
    found = True
    break
if found == False:
  print("Number not found")
```



2. Choose cricket team of eleven players find the captain of the team (consider tallest person as a captain) using dictionary

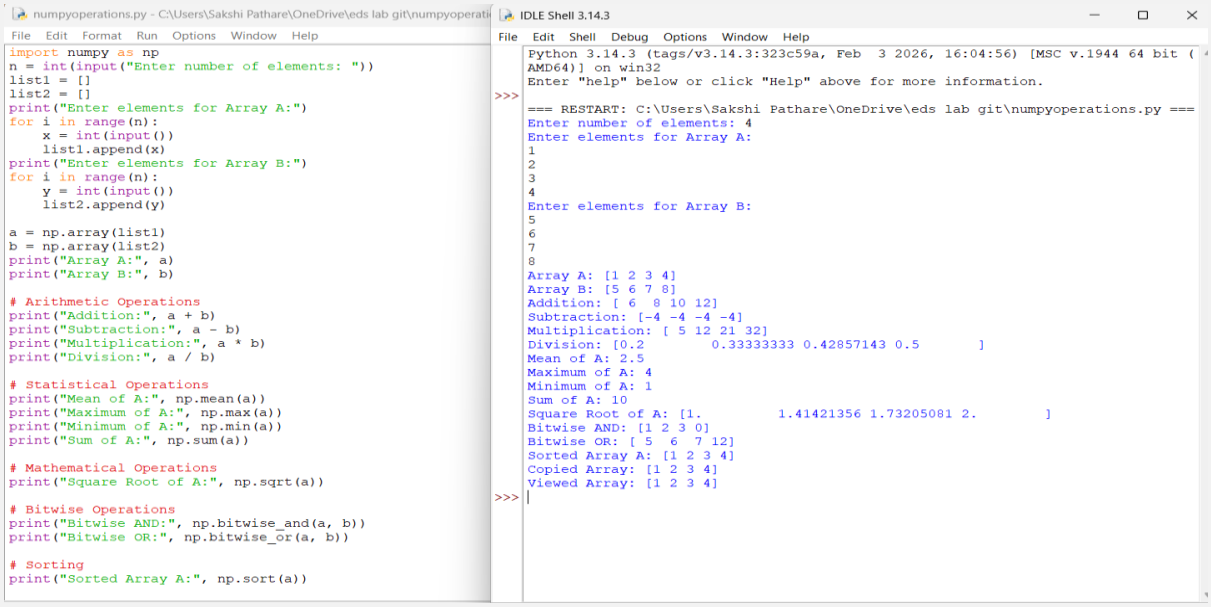
```
team = {}
n = int(input("Enter number of players: "))
for i in range(n):
    name = input("Enter player name: ")
    height = int(input("Enter height of player: "))
    team[name] = height
print("Team Dictionary:", team)
captain = max(team, key=team.get)
print("Captain of the team is:", captain)
print("Height:", team[captain], "cm")
```



PRACTICAL NO.03

Practice Lab Assignment:

1. Perform all the Numpy operations in python.



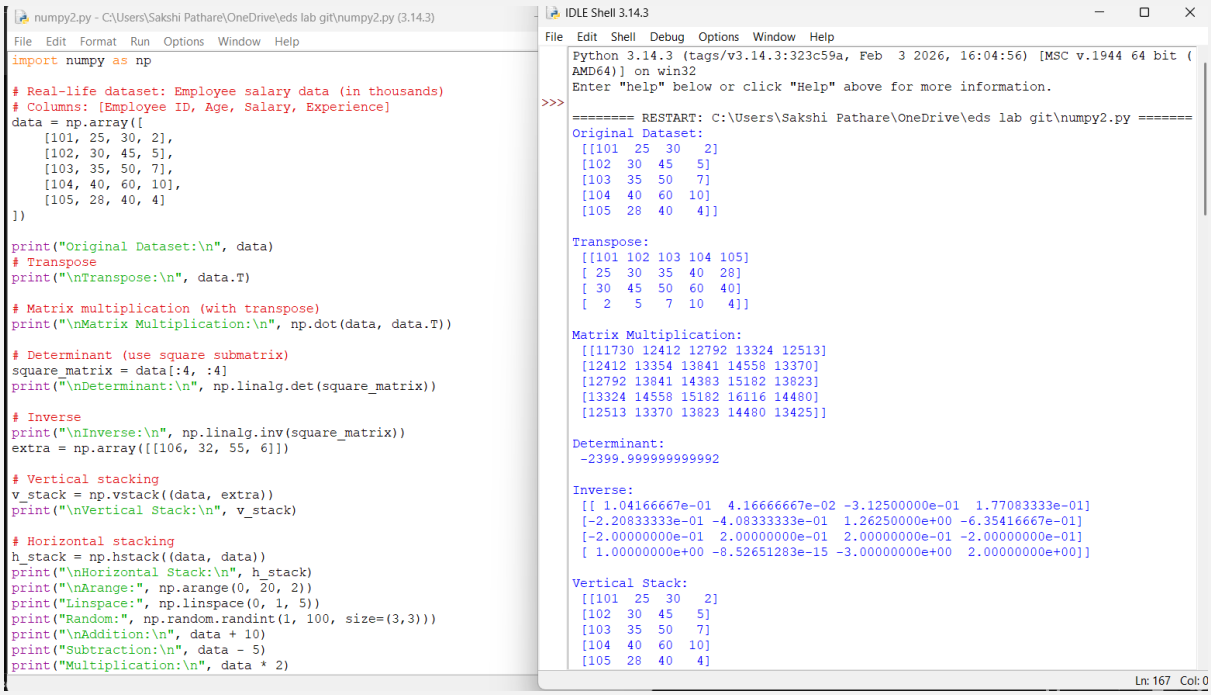
```
import numpy as np
n = int(input("Enter number of elements: "))
list1 = []
list2 = []
print("Enter elements for Array A:")
for i in range(n):
    x = int(input())
    list1.append(x)
print("Enter elements for Array B:")
for i in range(n):
    y = int(input())
    list2.append(y)

a = np.array(list1)
b = np.array(list2)
print("Array A:", a)
print("Array B:", b)

# Arithmetic Operations
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)

# Statistical Operations
print("Mean of A:", np.mean(a))
print("Maximum of A:", np.max(a))
print("Minimum of A:", np.min(a))
print("Sum of A:", np.sum(a))

# Mathematical Operations
print("Square Root of A:", np.sqrt(a))

# Bitwise Operations
print("Bitwise AND:", np.bitwise_and(a, b))
print("Bitwise OR:", np.bitwise_or(a, b))

# Sorting
print("Sorted Array A:", np.sort(a))
```

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:\Users\Sakshi Pathare\OneDrive\eds lab git\numpyoperations.py ===
Enter number of elements: 4
Enter elements for Array A:
1
2
3
4
Enter elements for Array B:
5
6
7
8
Array A: [1 2 3 4]
Array B: [5 6 7 8]
Addition: [ 6  8 10 12]
Subtraction: [-4 -4 -4 -4]
Multiplication: [ 5 12 21 32]
Division: [0.2        0.33333333 0.42857143 0.5       ]
Mean of A: 2.5
Maximum of A: 4
Minimum of A: 1
Sum of A: 10
Square Root of A: [1.         1.41421356 1.73205081 2.        ]
Bitwise AND: [1 2 3 0]
Bitwise OR: [ 5  6  7 12]
Sorted Array A: [1 2 3 4]
Copied Array: [1 2 3 4]
Viewed Array: [1 2 3 4]
>>>
```

Lab Assignment:

Prepare/Take dataset for any real life application. Read a dataset into an array. Perform following
operations on it as:

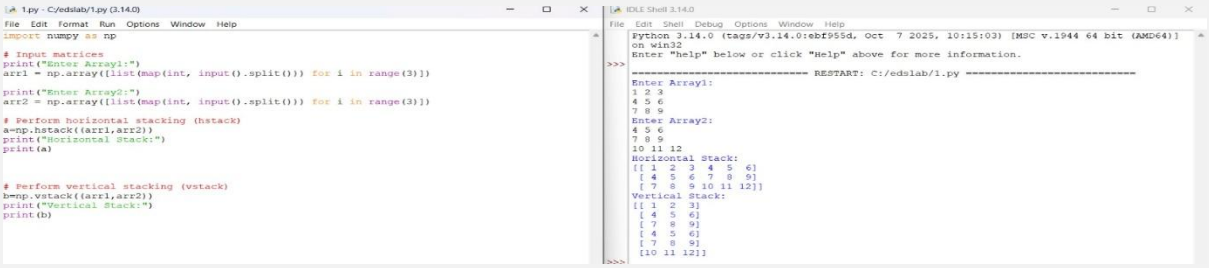
- Perform all matrix operations
- Horizontal and vertical stacking of Numpy Arrays
- Custom sequence generation
- Arithmetic and Statistical Operations, Mathematical Operations, Bitwise Operators
- Copying and viewing arrays
- Data Stacking, Searching, Sorting, Counting, Broadcasting



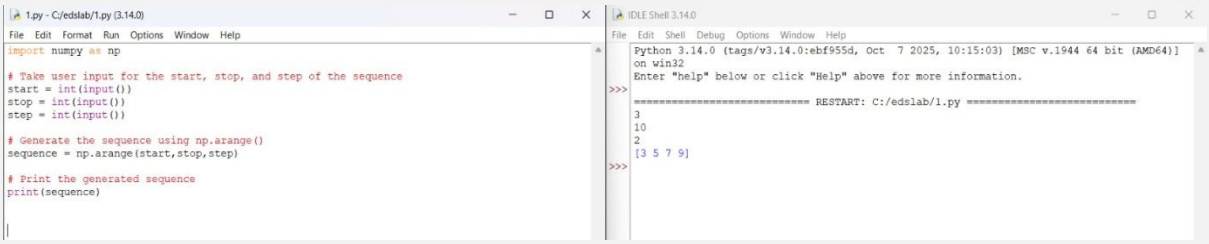
```
import numpy as np

# Real-life dataset: Employee salary data (in thousands)
# Columns: [Employee ID, Age, Salary, Experience]
data = np.array([
    [101, 25, 30, 2],
    [102, 30, 45, 5],
    [103, 35, 50, 7],
    [104, 40, 60, 10],
    [105, 28, 40, 4]
])

print("Original Dataset:\n", data)
# Transpose
print("\nTranspose:\n", data.T)

# Matrix multiplication (with transpose)
print("\nMatrix Multiplication:\n", np.dot(data, data.T))

# Determinant (use square submatrix)
square_matrix = data[:4, :4]
print("\nDeterminant:\n", np.linalg.det(square_matrix))

# Inverse
print("\nInverse:\n", np.linalg.inv(square_matrix))
extra = np.array([[106, 32, 55, 6]])

# Vertical stacking
v_stack = np.vstack((data, extra))
print("\nVertical Stack:\n", v_stack)

# Horizontal stacking
h_stack = np.hstack((data, data))
print("\nHorizontal Stack:\n", h_stack)
print("\nArange:", np.arange(0, 20, 2))
print("Linspace:", np.linspace(0, 1, 5))
print("Random:", np.random.randint(1, 100, size=(3,3)))
print("\nAddition:\n", data + 10)
print("Subtraction:\n", data - 5)
print("Multiplication:\n", data * 2)
```

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
======== RESTART: C:\Users\Sakshi Pathare\OneDrive\eds lab git\numpy2.py =======
Original Dataset:
 [[101  25  30   2]
 [102  30  45   5]
 [103  35  50   7]
 [104  40  60  10]
 [105  28  40   4]]

Transpose:
 [[101 102 103 104 105]
 [ 25  30  35  40  28]
 [ 30  45  50  60  40]
 [  2   5   7  10   4]]

Matrix Multiplication:
 [[11730 12412 12792 13324 12513]
 [12412 13354 13841 14558 13370]
 [12792 13841 14383 15182 13823]
 [13324 14558 15182 16116 14480]
 [12513 13370 13823 14480 13425]]

Determinant:
 -2399.999999999992

Inverse:
 [[ 1.04166667e-01  4.16666667e-02 -3.12500000e-01  1.77083333e-01]
 [-2.20833333e-01 -4.08333333e-01  1.26250000e+00 -6.35416667e-01]
 [-2.00000000e-01  2.00000000e-01  2.00000000e-01 -2.00000000e-01]
 [ 1.00000000e+00 -8.52651283e-15 -3.00000000e+00  2.00000000e+00]]

Vertical Stack:
 [[101  25  30   2]
 [102  30  45   5]
 [103  35  50   7]
 [104  40  60  10]
 [105  28  40   4]
```

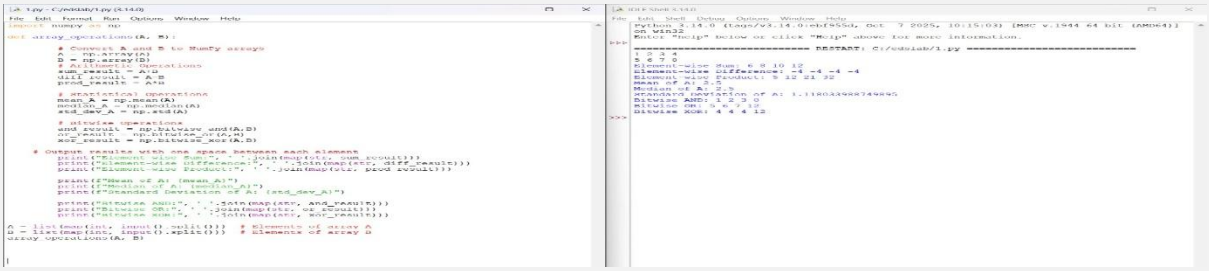
\



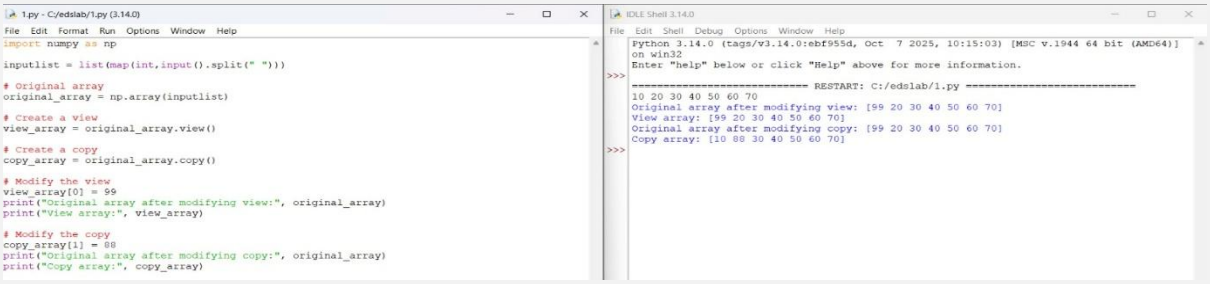
- Arithmetic and Statistical Operations



- Bitwise Operators

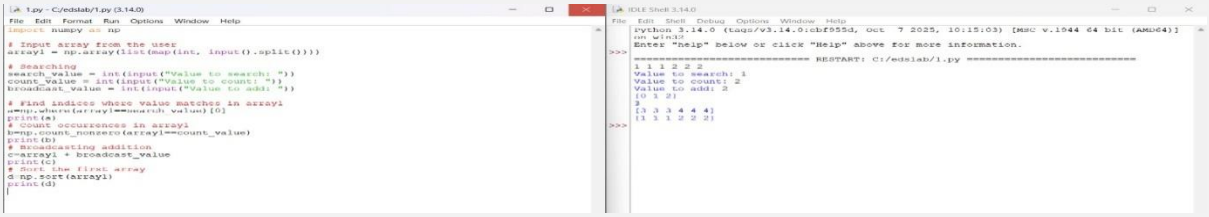


- Copying and viewing arrays



- Data Stacking
- Searching
- Sorting

- Broadcasting
- Counting

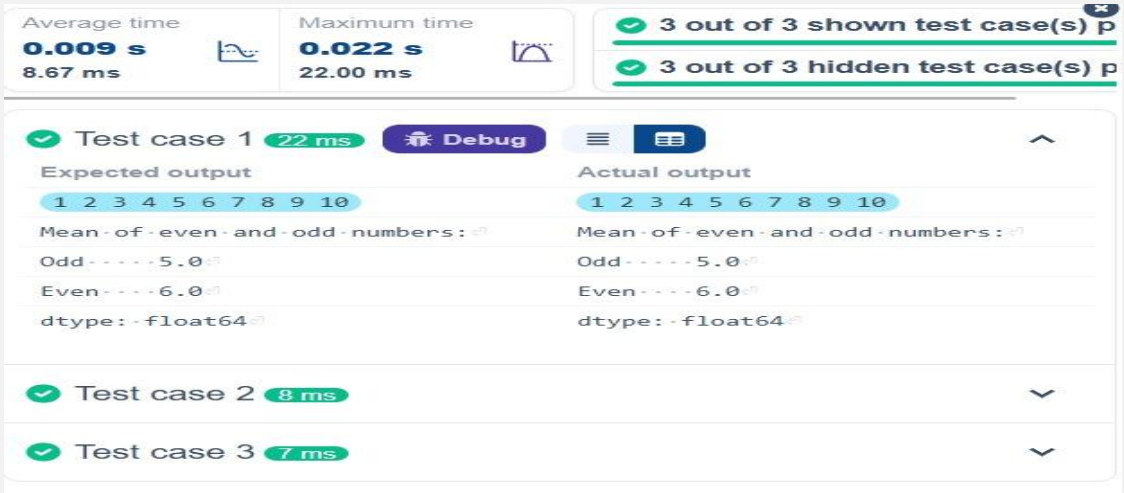


PRACTICAL NO. 04

Practice Lab Assignment:

1. Perform all the pandas operations in Python.

```
import pandas as pd

# Take inputs from the user to create a list of
numbers
numbers = list(map(int, input().split()))

# Create a Pandas series from the list of numbers
series = pd.Series(numbers)
# Grouping by even and odd numbers and calculating
the mean
grouped = series.groupby(series % 2 == 0).mean()

# Display the mean of even and odd numbers with
labels
grouped.index = ['Even' if is_even else 'Odd' for
is_even in grouped.index]
print("Mean of even and odd numbers:")
print(grouped)
```

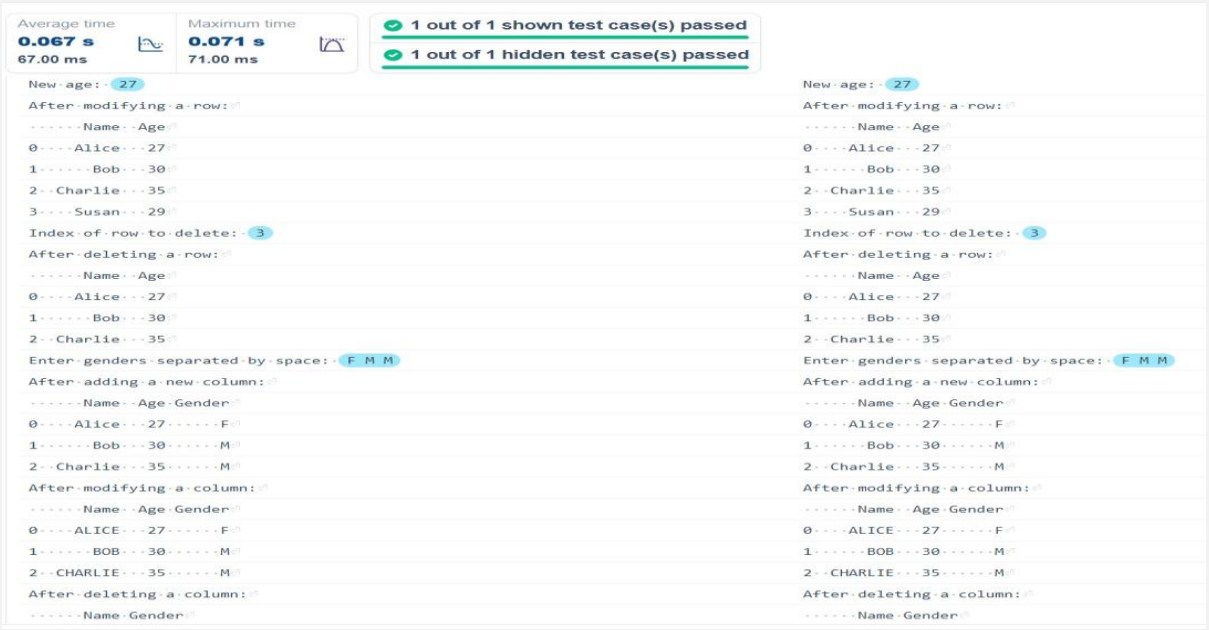


Lab Assignment:

Read any real life dataset. Store the data into Data Frames. Identify 10 grains for the given dataset. Implement all 20 grains using Pandas methods.

```
# Deleting a row
delt=int(input("Index of row to delete: "))
df=df.drop(delt).reset_index(drop=True)
# Display the DataFrame after deleting a row
print("After deleting a row:")
print(df)

# Adding a new column
genders = input("Enter genders separated by space: ").split()
df["Gender"] = genders

# Display the DataFrame after adding a new column
print("After adding a new column:")
print(df)

# Modifying a column
df['Name']=df['Name'].str.upper()
# Display the DataFrame after modifying a column
print("After modifying a column:")
print(df)

# Deleting a column
df=df.drop(columns=['Age'])
# Display the DataFrame after deleting a column
print("After deleting a column:")
print(df)
```

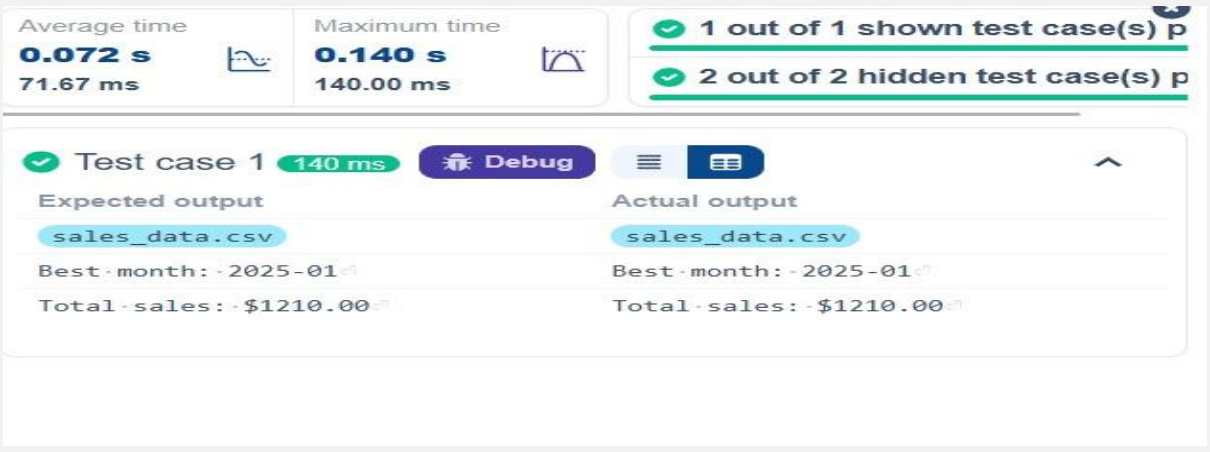
```
import pandas as pd

# Provided dictionary of lists
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
}

# Convert the dictionary to a DataFrame
df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Adding a new row
Name=input("New name: ")
Age=int(input("New age: "))
new={'Name':Name,'Age':Age}
df=df.append(new,ignore_index=True)

# Display the DataFrame after adding a new row
print("After adding a row:\n",df)

# Modifying a row
index=int(input("Index of row to modify: "))
age=int(input("New age: "))
df.at[index,"Age"]=age
```



The Sample Grains for Sales Dataset as:

Which was the best month for sales? How much was earned that month?

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)
df['Date']=pd.to_datetime(df['Date'])
df['Month']=df['Date'].dt.to_period('M').astype(str
)

# Find the month with the highest total sales
df['Sales']=df['Quantity']*df["Price"]
monthly_sales=df.groupby('Month')['Sales'].sum()
best_month =monthly_sales.idxmax()
highest_sales = monthly_sales.max()

print(f"Best month: {best_month}")
print(f"Total sales: ${highest_sales:.2f}")
```



Which product sold the most? Why do you think it did?

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)
product_sales=df.groupby('Product')
['Quantity'].sum().reset_index()

# Find the product with the highest total quantity
sold
best_product
=product_sales.loc[product_sales['Quantity'].idxmax
(),'Product']
highest_quantity = product_sales['Quantity'].max()

# Display the result
print(f"Best selling product: {best_product}")
print(f"Total quantity sold: {highest_quantity}")
```

sales_data.csv

```
Best selling product: Product A
Total quantity sold: 23
```

Which city sold the most products?

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

city_wise_sales=df.groupby('City')['Quantity'].sum()

# write the code..
best_city= city_wise_sales.idxmax()
# Display the result
print(f"City sold the most products: {best_city}")
```

sales_data.csv

```
City sold the most products: Los Angeles
```

Which city sold the most products?

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

city_wise_sales=df.groupby('City')['Quantity'].sum()

# write the code..
best_city= city_wise_sales.idxmax()
# Display the result
print(f"City sold the most products: {best_city}")
```

sales_data.csv

```
City sold the most products: Los Angeles
```

What Products are most often sold together?

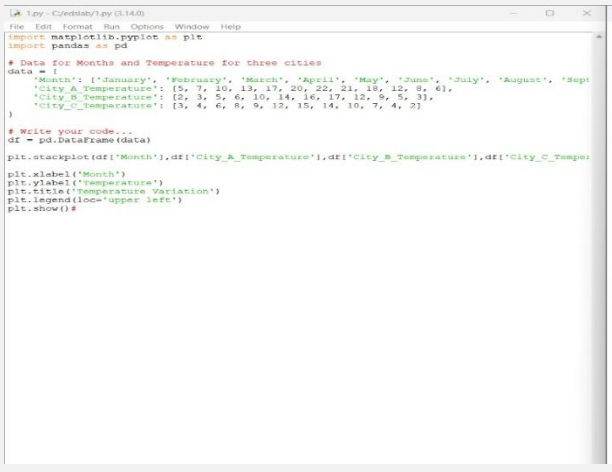
```
import pandas as pd
from itertools import combinations
from collections import Counter

# Prompt user to input the file name
file_name = input()

# Read data from the specified CSV file
df = pd.read_csv(file_name)
grouped = df.groupby("Date")["Product"].apply(list)
pairs_counter = Counter()
for products in grouped:
    pairs = combinations(sorted(products), 2)
    pairs_counter.update(pairs)
max_frequency = max(pairs_counter.values())
most_common_pairs = [(pair, freq) for pair , freq in
pairs_counter.items() if freq == max_frequency]
for (product1,product2), frequency in most_common_pairs:
    print(f"{product1} and {product2}: {frequency} times")
```

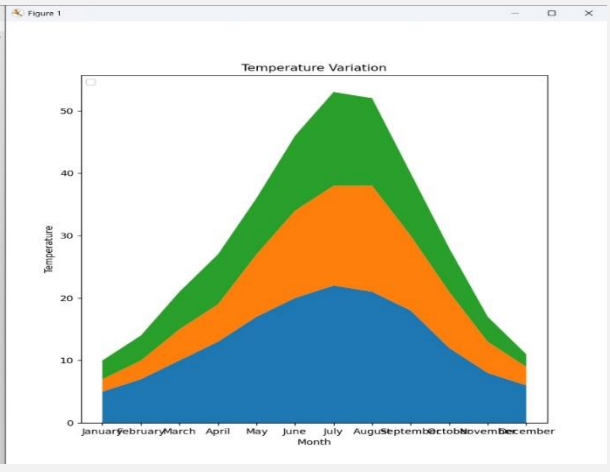
```
sales_data.csv
Product A and Product B: 2 times
Product A and Product C: 2 times
```

PRACTICAL NO. 05

Practice Lab Assignment:

Install MatPlotLib library. Draw basic graphs for sales dataset using MatPlotLib.





Lab Assignment:

For Titanic dataset, Perform data analysis. Identify 5 grains for a given dataset.

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset from the CSV file
df = pd.read_csv('titanic.csv')

# Set up the figure for 5 subplots
fig, axes = plt.subplots(3, 2, figsize=(12, 12))

# write the code.
# Plot 1: Count of passengers by class
axes[0,0].bar(df['Pclass'].value_counts().index,df['Pclass'].value_counts
(), color='skyblue')
axes [0, 0].set_title("Passenger Class Distribution")
axes [0, 0].set_xlabel("Pclass")
axes [0, 0].set_ylabel("Count")

# Plot 2: Gender distribution
axes[0,1].pie(df['Gender'].value_counts(),labels=df['Gender'].value_count
s().index, autopct='%1.1f%%', colors= ['lightblue', 'lightcoral'])
axes[0, 1].set_title("Gender Distribution")

# Plot 3: Age distribution
axes[1, 0].hist(df['Age'].dropna(), bins=8, color='lightgreen',
edgecolor='black')
axes[1, 0].set_title("Age Distribution")
axes[1, 0].set_xlabel("Age")
axes[1, 0].set_ylabel("Frequency")
```

```
# Plot 2: Gender distribution
axes[0,1].pie(df['Gender'].value_counts(),labels=df['Gender'].value_count
s().index, autopct='%1.1f%%', colors= ['lightblue', 'lightcoral'])
axes[0, 1].set_title("Gender Distribution")

# Plot 3: Age distribution
axes[1, 0].hist(df['Age'].dropna(), bins=8, color='lightgreen',
edgecolor='black')
axes[1, 0].set_title("Age Distribution")
axes[1, 0].set_xlabel("Age")
axes[1, 0].set_ylabel("Frequency")

# Plot 4: Survival count
axes[1, 1].bar(df['Survived'].value_counts().index,
df['Survived'].value_counts(), color=['lightblue', 'lightcoral'])
axes[1, 1].set_title("Survival Count")
axes[1, 1].set_xlabel("Survived (0 = No, 1 = Yes)")
axes[1, 1].set_ylabel("Count")

# Plot 5: Fare vs Age
axes[2, 0].scatter(df['Age'], df['Fare'], color='orange',
edgecolors='black')
axes[2, 0].set_title("Fare vs Age")
axes[2, 0].set_xlabel("Age")
axes[2, 0].set_ylabel("Fare")

plt.tight_layout()
plt.show()
```

Perform a data visualization for those grains using the MatPlotLib library.

(Use any 5 different graphs with proper title, legends, axis names, etc. to map identified grains)